ITA0612 – MACHINE LEARNING

CO5-AT1

PANDAS DATA ANALYSIS AND DATA PROCESSING

NAME: MONIKA R

REG.NO: 192324281

Submitted To:

Dr Shri Vindhya A

Submitted on:

21-08-2026

Question 1

Problem Statement

Two large Pandas DataFrames containing **customer transaction data** and **customer demographic data** must be combined. The operation must satisfy the following constraints:

1. No duplication of customer IDs.
2. Consistent indexing after merging.
3. Preservation of all valid records.
4. Validation using filtering and grouping.
5. Debugging of mismatched or missing customer entries.

Solution

The transaction DataFrame may contain information such as:

- Customer ID
- Transaction ID
- Product
- Amount
- Transaction Date

The demographic DataFrame may contain:

- Customer ID
- Name
- Age
- Gender
- Location
- Income

The common attribute between both DataFrames is **Customer_ID**. Therefore, the two DataFrames can be combined using the Pandas merge() operation.

Step 1: Import Pandas

```
import pandas as pd
```

Step 2: Load the Data

For example, assume the data is stored in CSV files.

```
transactions = pd.read_csv("transactions.csv")

demographics = pd.read_csv("demographics.csv")
```

For large datasets, unnecessary columns should not be loaded because they consume additional memory.

```
transactions = pd.read_csv(
    "transactions.csv",
    usecols=["Customer_ID", "Transaction_ID", "Product", "Amount", "Transaction_Date"]
)

demographics = pd.read_csv(
    "demographics.csv",
    usecols=["Customer_ID", "Name", "Age", "Gender", "Location", "Income"]
)
```

This improves memory efficiency and processing speed.

Step 3: Inspect the Data

Before merging, the structure of both DataFrames must be checked.

```
print(transactions.head())

print(demographics.head())

print(transactions.info())

print(demographics.info())
```

The number of rows and columns can also be checked.

```
print(transactions.shape)
```

```
print(demographics.shape)
```

This helps identify problems before performing the merge.

Step 4: Check Customer ID Duplicates

Since Customer_ID is the common key, duplicate customer IDs must be identified.

```
print(transactions["Customer_ID"].duplicated().sum())

print(demographics["Customer_ID"].duplicated().sum())
```

A customer may legitimately have multiple transactions, so duplicate IDs in the transaction table are not necessarily errors.

However, the demographic table should normally contain **one record per customer**.

```
duplicate_customers = demographics[
    demographics["Customer_ID"].duplicated(keep=False)
]

print(duplicate_customers)
```

If duplicate demographic records are found, they must be investigated.

Step 5: Remove Invalid Duplicate Demographic Records

If duplicate demographic records represent exactly the same customer information, they can be removed.

```
demographics = demographics.drop_duplicates(
    subset=["Customer_ID"],
    keep="first"
)
```

After this operation:

```
print(demographics["Customer_ID"].duplicated().sum())
```

The expected result should be:

```
0
```

Step 6: Check Missing Customer IDs

Missing customer IDs cannot be reliably used as merge keys.

```
print(transactions["Customer_ID"].isna().sum())

print(demographics["Customer_ID"].isna().sum())
```

Rows with missing customer IDs should be investigated.

```
missing_transaction_ids = transactions[

    transactions["Customer_ID"].isna()

]

missing_demographic_ids = demographics[

    demographics["Customer_ID"].isna()

]
```

If these records cannot be corrected, they may need to be separated for further data-quality processing rather than being silently deleted.

Step 7: Standardize Customer ID Format

A common reason for mismatched records is inconsistent formatting.

For example:

C001

c001

C001

C001

These may represent the same customer. Therefore, the IDs should be standardized.

```
transactions["Customer_ID"] = (

    transactions["Customer_ID"]

    .astype(str)

    .str.strip()
```

```
    .str.upper()
)
demographics["Customer_ID"] = (
    demographics["Customer_ID"]
    .astype(str)
    .str.strip()
    .str.upper()
)
```

Now both DataFrames use the same format.

Step 8: Find Mismatched Customer IDs

Customer IDs present in the transaction data but absent from demographic data can be identified using filtering.

```
missing_demographic = transactions[
    ~transactions["Customer_ID"].isin(demographics["Customer_ID"])
]
print(missing_demographic)
```

Similarly, customers present in demographic data but having no transaction can be identified.

```
missing_transactions = demographics[
    ~demographics["Customer_ID"].isin(transactions["Customer_ID"])
]
print(missing_transactions)
```

These checks are important for debugging incomplete datasets.

Step 9: Merge the DataFrames

Because all valid transaction records must be preserved, a **left join** is appropriate when transactions are considered the primary dataset.

```
merged_data = pd.merge(
    transactions,
    demographics,
    on="Customer_ID",
    how="left"
)
```

A left join ensures that every transaction remains in the final DataFrame even if demographic information is unavailable.

Step 10: Preserve All Valid Records

If the requirement is to preserve **all valid records from both datasets**, an outer join can be used.

```
merged_data = pd.merge(
    transactions,
    demographics,
    on="Customer_ID",
    how="outer",
    indicator=True
)
```

The _merge column shows where each record originated.

Possible values are:

both

left_only

right_only

Records with:

merged_data["_merge"] == "both"

are successfully matched.

Records with:

merged_data["_merge"] == "left_only"

exist only in the transaction dataset.

Records with:

merged_data["_merge"] == "right_only"

exist only in the demographic dataset.

Step 11: Validate the Merge

The number of records before and after merging should be compared.

print("Transactions:", len(transactions))

print("Demographics:", len(demographics))

print("Merged:", len(merged_data))

The merge should not unexpectedly multiply the number of rows.

This can happen when both DataFrames contain duplicate values for Customer_ID.

Step 12: Use Merge Validation

Pandas provides a useful validation feature.

If the demographic DataFrame contains one row per customer, the relationship is:

Many transactions → One demographic record

Therefore:

```
merged_data = pd.merge(
    transactions,
    demographics,
    on="Customer_ID",
    how="left",
    validate="many_to_one"
```

)

If duplicate customer IDs exist in the demographic table, Pandas raises an error instead of silently creating incorrect duplicated records.

This is an important debugging and data-quality technique.

Step 13: Reset the Index

After merging and filtering, the DataFrame index should be made consistent.

merged_data = merged_data.reset_index(drop=True)

The resulting index becomes:

0

1

2

3

...

This avoids inconsistent or duplicated index values.

Step 14: Validate Using Grouping

Customer-level transaction counts can be calculated using groupby().

```
customer_summary = (
    merged_data
    .groupby("Customer_ID")
    .agg(
        Transaction_Count=("Transaction_ID", "count"),
        Total_Amount=("Amount", "sum")
    )
    .reset_index()
)
```

This produces useful customer-level features for machine learning.

For example:

Customer_ID	Transaction_Count	Total_Amount
C001	5	12500
C002	2	4500
C003	8	19800

Such aggregated features can later be used for customer segmentation, churn prediction, or purchase prediction.

Step 15: Filter Invalid Records

Invalid transaction amounts can be identified.

```
invalid_amounts = merged_data[
    merged_data["Amount"] <= 0
]
print(invalid_amounts)
```

Missing demographic values can also be identified.

```
missing_demographics = merged_data[
    merged_data["Age"].isna() |
    merged_data["Gender"].isna() |
    merged_data["Location"].isna()
]
print(missing_demographics)
```

Step 16: Handle Missing Values

Missing numerical values can be handled using an appropriate strategy.

```
merged_data["Age"] = merged_data["Age"].fillna(
    merged_data["Age"].median()
)
```

```
merged_data["Income"] = merged_data["Income"].fillna(
    merged_data["Income"].median()
)
```

Categorical values can be filled using the mode.

```
merged_data["Gender"] = merged_data["Gender"].fillna(
    merged_data["Gender"].mode()[0]
)
```

The choice of missing-value strategy depends on the machine-learning problem and the nature of the data.

Step 17: Final Validation

Check for duplicate customer-demographic combinations.

```
print(
    merged_data[
        ["Customer_ID", "Name", "Age", "Gender"]
    ].duplicated().sum()
)
```

Check the final index:

```
print(merged_data.index.is_unique)
```

Check missing values:

```
print(merged_data.isnull().sum())
```

Check duplicate customer IDs in the demographic information:

```
print(
    demographics["Customer_ID"].duplicated().sum()
)
```

```
CSV FILES FOUND:
transactions1.csv
transactions3.csv
transactions2.csv
transactions1.csv loaded successfully - 5 rows
transactions3.csv loaded successfully - 5 rows
transactions2.csv loaded successfully - 5 rows

COMBINED DATA
   Customer ID  Amount  Age  Gender Transaction Date        Source_File
0         C001   55000   25    Male       2026-01-05  transactions1.csv
1         C002   30000   30  Female       2026-01-08  transactions1.csv
2         C003   25000   28    Male       2026-01-10  transactions1.csv
3         C004    5000   35  Female       2026-01-12  transactions1.csv
4         C005   15000   24    Male       2026-01-15  transactions1.csv
5         C011   18000   26    Male       2026-03-01  transactions3.csv
6         C012   60000   38  Female       2026-03-05  transactions3.csv
7         C013    7000   23    Male       2026-03-08  transactions3.csv
8         C014   28000   34  Female       2026-03-10  transactions3.csv
9         C015   40000   30    Male       2026-03-15  transactions3.csv
10        C006   12000   29  Female       2026-02-01  transactions2.csv
11        C007   45000   32    Male       2026-02-05  transactions2.csv
12        C008    8000   27  Female       2026-02-08  transactions2.csv
13        C009   22000   40    Male       2026-02-10  transactions2.csv
14        C010   35000   31  Female       2026-02-15  transactions2.csv

STANDARDIZED COLUMN NAMES:
['customer_id', 'amount', 'age', 'gender', 'transaction_date', 'source_file']

FIRST 5 RECORDS:
  customer_id  amount  age  gender transaction_date        source_file
0        C001   55000   25    Male       2026-01-05  transactions1.csv
1        C002   30000   30  Female       2026-01-08  transactions1.csv
2        C003   25000   28    Male       2026-01-10  transactions1.csv
3        C004    5000   35  Female       2026-01-12  transactions1.csv
4        C005   15000   24    Male       2026-01-15  transactions1.csv

DATASET SHAPE:
(15, 6)

DATA TYPES:
customer_id         object
amount               int64
age                  int64
gender              object
transaction_date    object
source_file         object
dtype: object
```

CUSTOMERS SORTED BY TOTAL SPENDING

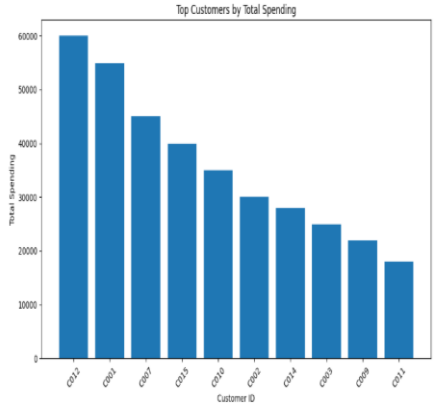
	customer_id	Total_Spending	Average_Spending	Transaction_Count	\
11	C012	60000	60000.0	1	
0	C001	55000	55000.0	1	
6	C007	45000	45000.0	1	
14	C015	40000	40000.0	1	
9	C010	35000	35000.0	1	
1	C002	30000	30000.0	1	
13	C014	28000	28000.0	1	
2	C003	25000	25000.0	1	
8	C009	22000	22000.0	1	
10	C011	18000	18000.0	1	
4	C005	15000	15000.0	1	
5	C006	12000	12000.0	1	
7	C008	8000	8000.0	1	
12	C013	7000	7000.0	1	
3	C004	5000	5000.0	1	

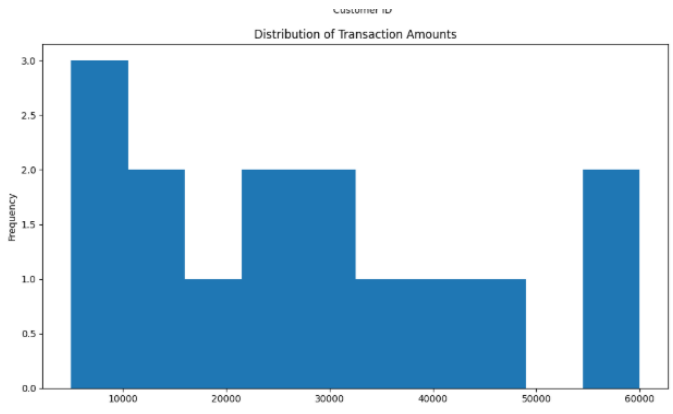
	Average_Transaction_Value
11	60000.0
0	55000.0
6	45000.0
14	40000.0
9	35000.0
1	30000.0
13	28000.0
2	25000.0
8	22000.0
10	18000.0
4	15000.0
5	12000.0
7	8000.0
12	7000.0
3	5000.0

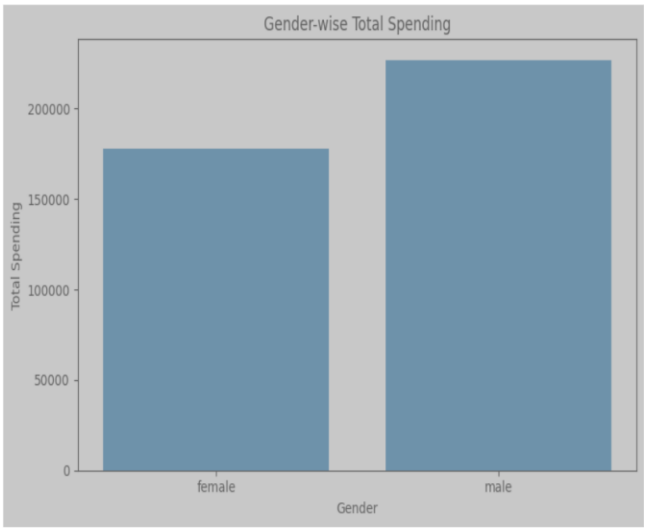
TOP 10 CUSTOMERS

	customer_id	Total_Spending	Average_Spending	Transaction_Count	\
11	C012	60000	60000.0	1	
0	C001	55000	55000.0	1	
6	C007	45000	45000.0	1	
14	C015	40000	40000.0	1	
9	C010	35000	35000.0	1	
1	C002	30000	30000.0	1	
13	C014	28000	28000.0	1	
2	C003	25000	25000.0	1	
8	C009	22000	22000.0	1	
10	C011	18000	18000.0	1	

	Average_Transaction_Value
11	60000.0
0	55000.0
6	45000.0
14	40000.0
9	35000.0
1	30000.0
13	28000.0







---------- FINAL VALIDATION ----------
Final Dataset Shape:
(15, 6)

Duplicate Records:
0

Missing Values:
customer_id 0
amount 0
age 0
gender 0
transaction_date 0
source_file 0
dtype: int64

Data Types:
customer_id object
amount int64
age int64
gender object
transaction_date datetime64[ns]
source_file object
dtype: object

Final Dataset:

	customer_id	amount	age	gender	transaction_date	source_file
0	C001	55000	25	male	2026-01-05	transactions1.csv
1	C002	30000	30	female	2026-01-08	transactions1.csv
2	C003	25000	28	male	2026-01-10	transactions1.csv
3	C004	5000	35	female	2026-01-12	transactions1.csv
4	C005	15000	24	male	2026-01-15	transactions1.csv
5	C011	18000	26	male	2026-03-01	transactions3.csv
6	C012	60000	38	female	2026-03-05	transactions3.csv
7	C013	7000	23	male	2026-03-08	transactions3.csv
8	C014	28000	34	female	2026-03-10	transactions3.csv
9	C015	40000	30	male	2026-03-15	transactions3.csv
10	C006	12000	29	female	2026-02-01	transactions2.csv
11	C007	45000	32	male	2026-02-05	transactions2.csv
12	C008	8000	27	female	2026-02-08	transactions2.csv
13	C009	22000	40	male	2026-02-10	transactions2.csv
14	C010	35000	31	female	2026-02-15	transactions2.csv

Cleaned dataset saved to:
/content/cleaned_ml_dataset.csv

Debugging Strategy

The following procedure can be used when mismatched or missing records occur:

1. Check whether Customer_ID contains null values.
2. Check whether IDs have leading/trailing spaces.
3. Convert IDs to a common data type.
4. Convert IDs to a common case using uppercase/lowercase.
5. Check duplicate customer IDs.
6. Compare unique IDs in both datasets.
7. Use isin() to find unmatched IDs.
8. Use an outer merge with indicator=True to identify the source of unmatched records.
9. Check whether the merge relationship is correct using validate.
10. Reset the index after all transformations.
11. Recheck row counts and missing values.

Complete Example

```
import pandas as pd

# Load data

transactions = pd.read_csv("transactions.csv")

demographics = pd.read_csv("demographics.csv")

# Standardize Customer_ID

transactions["Customer_ID"] = (

    transactions["Customer_ID"]

    .astype(str)

    .str.strip()

    .str.upper()

)
```

```
demographics["Customer_ID"] = (
    demographics["Customer_ID"]
    .astype(str)
    .str.strip()
    .str.upper()
)
demographics = demographics.drop_duplicates(
    subset=["Customer_ID"]
)
missing_demo = transactions[
    ~transactions["Customer_ID"].isin(
        demographics["Customer_ID"]
    )
]
print("Customers without demographic data:")
print(missing_demo)
merged_data = pd.merge(
    transactions,
    demographics,
    on="Customer_ID",
    how="left",
    validate="many_to_one"
)
# Reset index
merged_data = merged_data.reset_index(drop=True)
```

```
# Grouping
customer_summary = (
    merged_data
    .groupby("Customer_ID")
    .agg(
        Transaction_Count=("Transaction_ID", "count"),
        Total_Amount=("Amount", "sum")
    )
    .reset_index()
)
print("Merged shape:", merged_data.shape)
print("Duplicate rows:", merged_data.duplicated().sum())
print("Missing values:")
print(merged_data.isnull().sum())
print("\nCustomer Summary:")
print(customer_summary.head())
```

Conclusion

The two DataFrames can be safely combined using Pandas merge(). Before merging, customer IDs must be standardized, missing IDs identified, and duplicate demographic records removed. A left join preserves all transaction records, while an outer join can be used when records from both datasets must be preserved. Filtering with isin(), grouping with groupby(), merge indicators, and validate="many_to_one" help detect inconsistencies and prevent incorrect data duplication. Finally, resetting the index ensures consistent indexing. The resulting clean dataset can be used as input for machine-learning tasks such as customer segmentation, churn prediction, and customer-value prediction.

Question 2

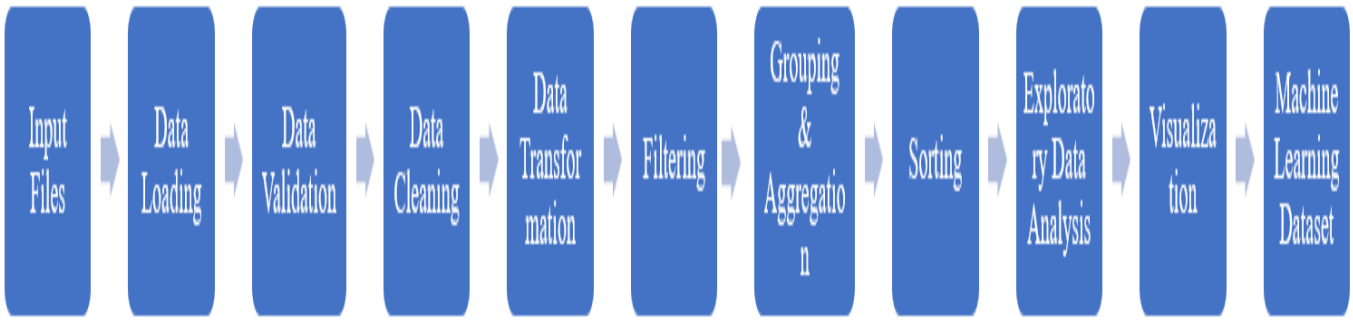
Problem Statement

Design a complete data analysis pipeline that:

1. Reads multiple data files.
2. Processes the files using Pandas DataFrames.
3. Performs efficient file I/O.
4. Groups and sorts the data correctly.
5. Filters records according to user-defined conditions.
6. Performs aggregation.
7. Generates meaningful visualizations.
8. Handles corrupted files and inconsistent data formats.
9. Produces data suitable for machine-learning analysis.

Solution

A complete machine-learning data-analysis pipeline can be represented as:



Stage 1: Import Required Libraries

```
import pandas as pd

import matplotlib.pyplot as plt

from pathlib import Path
```

Pandas is used for data manipulation and analysis, while Matplotlib is used for visualization.

Stage 2: Identify Multiple Data Files

If multiple CSV files are stored in a directory:

```
data_path = Path("data")

files = list(data_path.glob("*.csv"))

print(files)
```

This automatically identifies all CSV files instead of manually specifying each file.

Stage 3: Efficient File Loading

The files can be read using a loop.

```
dataframes = []

for file in files:
    try:
        df = pd.read_csv(file)
        df["Source_File"] = file.name
        dataframes.append(df)
    except Exception as e:
        print(f"Error reading {file}: {e}")
```

This approach prevents one corrupted file from stopping the entire pipeline.

Stage 4: Combine Multiple Files

After successfully loading the files:

```
data = pd.concat(
    dataframes,
    ignore_index=True
)
```

ignore_index=True creates a new continuous index.

Stage 5: Inspect the Dataset

```
print(data.head())

print(data.shape)

print(data.info())

print(data.describe())
```

These operations help understand:

- Number of records
- Number of features
- Data types
- Missing values
- Numerical distributions
- Possible outliers

Stage 6: Handle Inconsistent Column Names

Different files may contain inconsistent column names.

For example:

Customer ID

customer_id

CustomerID

These should be standardized.

```
data.columns = (
    data.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
```

Now the columns follow a consistent naming convention.

Stage 7: Remove Duplicate Records

Duplicate records can negatively affect machine-learning models.

```
print("Duplicates:", data.duplicated().sum())

data = data.drop_duplicates()
```

The data should then be checked again.

```
print("Duplicates after cleaning:",
      data.duplicated().sum())
```

Stage 8: Handle Missing Values

Missing values can be detected using:

```
print(data.isnull().sum())
```

Numerical columns can use median imputation:

```
numeric_columns = data.select_dtypes(
    include="number"
).columns

data[numeric_columns] = data[numeric_columns].fillna(
    data[numeric_columns].median()
)
```

Categorical columns can use the mode:

```
categorical_columns = data.select_dtypes(
    include="object"
).columns

for column in categorical_columns:
    if data[column].isnull().any():
        data[column] = data[column].fillna(
```

```
        data[column].mode()[0]
    )
```

The appropriate strategy should depend on the dataset and ML model.

Stage 9: Convert Data Types

Suppose the transaction amount is stored as text.

```
data["amount"] = pd.to_numeric(
    data["amount"],
    errors="coerce"
)
```

Date columns should also be converted.

```
data["transaction_date"] = pd.to_datetime(
    data["transaction_date"],
    errors="coerce"
)
```

errors="coerce" converts invalid values into NaN/NaT, which can then be identified and handled.

Stage 10: Filter Data

Filtering allows the analyst to select records according to specific conditions.

For example, customers whose transaction amount is greater than 10,000:

```
high_value = data[
    data["amount"] > 10000
]
```

Multiple conditions can be applied:

```
filtered_data = data[
    (data["amount"] > 5000) &
```

```
    (data["age"] >= 18)
]
```

Filtering reduces unnecessary records and allows focused analysis.

Stage 11: Grouping

Grouping can be used to calculate customer-level statistics.

```
customer_analysis = (
    data
    .groupby("customer_id")
    .agg(
        Total_Spending=("amount", "sum"),
        Average_Spending=("amount", "mean"),
        Transaction_Count=("amount", "count")
    )
    .reset_index()
)
```

This converts transaction-level data into customer-level features.

These features can later be used by machine-learning algorithms.

Stage 12: Sorting

The aggregated records can be sorted according to total spending.

```
customer_analysis = customer_analysis.sort_values(
    by="Total_Spending",
    ascending=False
)
```

The highest-value customers will appear first.

Stage 13: Create Machine-Learning Features

Feature engineering is an important stage of the machine-learning pipeline.

For example:

```
customer_analysis["Average_Transaction_Value"] = (
    customer_analysis["Total_Spending"] /
    customer_analysis["Transaction_Count"]
)
```

Other useful features may include:

- Total spending
- Number of transactions
- Average transaction value
- Purchase frequency
- Customer age
- Income
- Geographic location
- Product category

Feature engineering improves the usefulness of the dataset for machine-learning models.

Stage 14: Generate Visualization

A meaningful visualization can show the top customers based on spending.

```
top_customers = customer_analysis.head(10)
plt.figure(figsize=(10, 6))
plt.bar(
    top_customers["customer_id"].astype(str),
    top_customers["Total_Spending"]
)
```

```
plt.xlabel("Customer ID")
plt.ylabel("Total Spending")
plt.title("Top 10 Customers by Total Spending")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

This visualization helps identify high-value customers.

Stage 15: Distribution Visualization

A histogram can be used to understand the distribution of transaction amounts.

```
plt.figure(figsize=(10, 6))
plt.hist(
    data["amount"].dropna(),
    bins=20
)
plt.xlabel("Transaction Amount")
plt.ylabel("Frequency")
plt.title("Distribution of Transaction Amounts")
plt.tight_layout()
plt.show()
```

This can help identify:

- Typical transaction values
- Skewed distributions
- Outliers
- Unusual transaction behavior

Stage 16: Efficient I/O Optimization

For very large datasets, loading the entire dataset into memory may be inefficient.

Only required columns should be loaded.

```
df = pd.read_csv(
    "transactions.csv",
    usecols=[
        "customer_id",
        "amount",
        "transaction_date"
    ]
)
```

Data types can also be specified.

```
df = pd.read_csv(
    "transactions.csv",
    dtype={
        "customer_id": "string",
        "amount": "float32"
    }
)
```

For extremely large CSV files, chunk processing can be used.

```
for chunk in pd.read_csv(
    "transactions.csv",
    chunksize=10000
):
    process_chunk(chunk)
```

Chunk processing prevents the entire dataset from being loaded into memory at once.

Debugging Corrupted Files

A robust pipeline should not fail completely when one input file is corrupted.

```
for file in files:
    try:
        df = pd.read_csv(file)
        print(
            f"{file.name}: "
            f"{df.shape[0]} rows loaded"
        )
    except pd.errors.ParserError:
        print(
            f"Parser error in {file.name}"
        )
    except UnicodeDecodeError:
        print(
            f"Encoding problem in {file.name}"
        )
    except Exception as e:
        print(
            f"Unexpected error in {file.name}: {e}"
        )
```

This helps identify problematic files.

Handling Inconsistent Data Formats

Different files may use different formats.

For example:

2026-01-15

15/01/2026

01-15-2026

These can be converted into a common date format:

```
data["transaction_date"] = pd.to_datetime(
    data["transaction_date"],
    errors="coerce"
)
```

Similarly, categorical values should be standardized.

For example:

Male

male

M

can be converted into a common representation.

```
data["gender"] = (
    data["gender"]
    .astype(str)
    .str.strip()
    .str.lower()
)
```

Final Data Validation

Before using the dataset for machine learning, several checks should be performed.

Check missing values

```
print(data.isnull().sum())
```

Check duplicates

```
print(data.duplicated().sum())
```

Check data types

```
print(data.dtypes)
```

Check numerical statistics

```
print(data.describe())
```

Check unique categorical values

```
print(data["gender"].unique())
```

Check final shape

```
print(data.shape)
```

These checks ensure that the dataset is suitable for further analysis.

Complete Pipeline Example

```
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
# 1. Locate files
data_path = Path("data")
files = list(data_path.glob("*.csv"))
dataframes = []
# 2. Load files
for file in files:
    try:
        df = pd.read_csv(file)
        df["source_file"] = file.name
       dataframes.append(df)
```

```
    except pd.errors.ParserError:
        print(f"Corrupted CSV: {file}")
    except Exception as e:
        print(f"Error in {file}: {e}")
# 3. Combine files
data = pd.concat(
    dataframes,
    ignore_index=True
)
# 4. Standardize column names
data.columns = (
    data.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
# 5. Remove duplicates
data = data.drop_duplicates()
# 6. Convert numeric column
if "amount" in data.columns:
    data["amount"] = pd.to_numeric(
        data["amount"],
        errors="coerce"
    )
```

```
# 7. Convert date
if "transaction_date" in data.columns:
    data["transaction_date"] = pd.to_datetime(
        data["transaction_date"],
        errors="coerce"
    )
# 8. Remove invalid amounts
if "amount" in data.columns:
    data = data[data["amount"] > 0]
# 9. Group and aggregate
if "customer_id" in data.columns:
    customer_analysis = (
        data
        .groupby("customer_id")
        .agg(
            Total_Spending=("amount", "sum"),
            Average_Spending=("amount", "mean"),
            Transaction_Count=("amount", "count")
        )
        .reset_index()
    )
    # 10. Sort
    customer_analysis = customer_analysis.sort_values(
        "Total_Spending",
        ascending=False
```

```
    )
    # 11. Display result
    print(customer_analysis.head(10))
    # 12. Visualization
    top = customer_analysis.head(10)
    plt.figure(figsize=(10, 6))
    plt.bar(
        top["customer_id"].astype(str),
        top["Total_Spending"]
    )
    plt.xlabel("Customer ID")
    plt.ylabel("Total Spending")
    plt.title("Top 10 Customers by Total Spending")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
# 13. Final validation
print("Final dataset shape:", data.shape)
print("Duplicate records:", data.duplicated().sum())
print("Missing values:")
print(data.isnull().sum())
```

Machine Learning Relevance

The complete pipeline is directly useful in machine learning because raw data normally cannot be supplied directly to an ML algorithm. It must first be cleaned, transformed, and converted into meaningful features.

The process can be summarized as:

Raw Data

↓

Data Loading

↓

Data Cleaning

↓

Missing Value Handling

↓

Duplicate Removal

↓

Data Transformation

↓

Feature Engineering

↓

Filtering

↓

Grouping & Aggregation

↓

Exploratory Data Analysis

↓

Visualization

↓

Feature Dataset

↓

Machine Learning Model

For example, customer transaction data can be transformed into features such as:

Customer_ID

Total_Spending

Transaction_Count

Average_Transaction_Value

Age

Income

Location

These features can be used for machine-learning applications such as:

- Customer segmentation using **K-Means clustering**
- Customer churn prediction
- Customer lifetime value prediction
- Purchase prediction
- Fraud detection
- Recommendation systems

Conclusion

A complete Pandas-based data analysis pipeline should begin with efficient loading of multiple files and continue through validation, cleaning, transformation, filtering, grouping, aggregation, sorting, and visualization. Error handling should be included to identify corrupted files and inconsistent formats without terminating the entire process. The final cleaned and engineered dataset can then be used as input for machine-learning algorithms. Pandas provides efficient functions such as read_csv(), concat(), merge(), groupby(), agg(), sort_values(), filtering, and missing-value handling, making it suitable for building scalable data preprocessing pipelines.